

```

# Install required libraries
!pip install tensorflow keras opencv-python pillow textblob numpy --
quiet
!python -m textblob.download_corpora

# Upload dataset ZIP (fer_dataset.zip containing train/happy,
train/sad)
from google.colab import files
uploaded = files.upload() # Upload your dataset zip file here, named
"fer_dataset.zip"

# Unzip dataset
import zipfile
with zipfile.ZipFile('fer_dataset.zip', 'r') as zip_ref:
    zip_ref.extractall('fer_dataset')

# Organize only 'happy' and 'sad' images into Positive & Negative
folders
import os
import shutil

# Create target folders
os.makedirs('dataset/Positive', exist_ok=True)
os.makedirs('dataset/Negative', exist_ok=True)

# Move happy images to Positive
happy_dir = 'fer_dataset/train/happy'
for img_file in os.listdir(happy_dir):
    shutil.move(os.path.join(happy_dir, img_file),
'dataset/Positive/')

# Move sad images to Negative
sad_dir = 'fer_dataset/train/sad'
for img_file in os.listdir(sad_dir):
    shutil.move(os.path.join(sad_dir, img_file), 'dataset/Negative/')

# Image Sentiment Model Training
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Data Preprocessing
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_data = datagen.flow_from_directory(
    'dataset',
    target_size=(64, 64),
    batch_size=32,

```

```

        class_mode='binary',
        subset='training'
    )

val_data = datagen.flow_from_directory(
    'dataset',
    target_size=(64, 64),
    batch_size=32,
    class_mode='binary',
    subset='validation'
)

# CNN Model
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64, 64, 3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

# Compile
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train
model.fit(train_data, epochs=15, validation_data=val_data)

# Save model
model.save("fer_image_sentiment_model.h5")
print("✅ Model trained and saved.")

# 📁 Upload an image for prediction
uploaded = files.upload() # Upload your test image

# 📁 Image Sentiment Prediction Function
import numpy as np
from PIL import Image
from tensorflow.keras.models import load_model

def predict_image_sentiment(image_path):
    model = load_model("fer_image_sentiment_model.h5")
    img = Image.open(image_path).convert('RGB').resize((64, 64))
    img_array = np.array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)[0][0]

```

```

    sentiment = 'Positive 😊' if prediction > 0.5 else 'Negative 😞'
    print(f"🖼 Image Sentiment: {sentiment} (Confidence:
{prediction:.2f})")

# 📄 Text Sentiment Analysis Function
from textblob import TextBlob

def predict_text_sentiment(text):
    analysis = TextBlob(text)
    polarity = analysis.sentiment.polarity
    sentiment = 'Positive 😊' if polarity > 0 else 'Negative 😞'
    print(f"📄 Text Sentiment: {sentiment} (Polarity: {polarity:.2f})")

# 📄 Combined Multimodal Sentiment Prediction
def multimodal_sentiment(image_path, text_input):
    predict_image_sentiment(image_path)
    predict_text_sentiment(text_input)

# 📄 Run Combined Test
# Replace 'your_image.jpg' with your actual uploaded image filename,
# and enter your test text
multimodal_sentiment('your_uploaded_image.jpg', "I'm feeling very
happy today!")

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Package brown is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package conll2000 to /root/nltk_data...
[nltk_data] Package conll2000 is already up-to-date!
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data] Package movie_reviews is already up-to-date!
Finished.

<IPython.core.display.HTML object>

```